# co-founder Blueprint v0.1 (Hackathon MVP → Product Foundation)

## 0) 목표 정의

- **목표(해커톤 MVP):** 회의 음성 입력을 받아 **실시간으로**
  1. co-founder가 “참가자처럼” 개입(반론/대안/질문)하고
  2. 동시에 **전략/아키텍처 구조를 시각화 캔버스**에 생성하며
  3. **2~3개 시나리오 비교**를 즉시 보여준다. Gemini Live Agent Challenge: Re...
- **목표(장기):** Conversation → World State → Simulation → Decision Memory로 확장 가능한 OS.

---

## 1) 규정 기반 필수 요구사항 (Fail 조건 제거 체크리스트)

해커톤 제출은 아래를 **반드시** 충족해야 한다.
Gemini Live Agent Challenge: Re...

### Mandatory

☐ **NEW Project** (기간 내 신규 제작, 기존 작업의 단순 확장/변형 금지) Gemini Live Agent Challenge: Re...

☐ **Gemini 모델 사용** Gemini Live Agent Challenge: Re...

☐ **Google GenAI SDK 또는 ADK로 에이전트 구축** Gemini Live Agent Challenge: Re...

☐ **Google Cloud 서비스 최소 1개 사용 + 백엔드가 Google Cloud에서 동작** Gemini Live Agent Challenge: Re...

☐ 제출물 5종:

- 텍스트 설명
- Public repo + README spin-up
- GCP 배포 증빙(데모와 별도 영상/또는 코드 증거)

- 아키텍처 다이어그램
- 4분 이내 실시간 작동 데모 영상(목업 금지) Gemini Live Agent Challenge: Re...

---

# 2) 제품 아키텍처(재사용 가능한 “5-레이어”)

> **규칙:** 레이어 간 책임 분리. LLM은 “출력 생성기”가 아니라 “World State 갱신기”.

## L1. Input Layer (Meeting I/O)

- 입력: mic audio → streaming transcript
- 출력: TTS(또는 audio playback) / 화면상 발화 큐
- 산출: `TranscriptEvent[]` (타임스탬프, 화자, confidence)

## L2. Interpretation Layer (World Compiler)

- 기능: transcript → 구조화된 **World State**로 컴파일
- 산출: `WorldState vN` (graph + assumptions + metrics)

## L3. Reasoning/Agent Layer (co-founder behavior)

- 기능: 참가자 규칙(언제 끼어들지, 무엇을 반론할지, 어떤 대안을 강제할지)
- 산출: `InterruptionPlan`, `Counterfactuals`, `QuestionQueue`

## L4. Visualization Layer (Live Canvas)

- 기능: World State를 시각화로 렌더링(다이어그램/트리/그래프)
- 산출: `mermaid/graph spec`, `scenario diff view`

## L5. Decision Memory Layer (Evidence & Replay)

- 기능: 회의 중 결정의 전제/대안/버린 안/채택 이유 자동 기록
- 산출: `DecisionLog` + `Replayable Session`

---

# 3) 핵심 데이터 모델 (MVP 최소 스키마)

**WorldState**를 “모든 것의 단일 진실”로 두면, 이후 확장이 직선이다.

## 3.1 WorldState (JSON)

- `session_id`
- `entities[]` (사람/프로젝트/시장/리소스)
- `assumptions[]` (가정, 근거, 신뢰도)
- `constraints[]` (예산/시간/정책)
- `options[]` (전략/아키텍처 선택지)
- `scenarios[]` (base/best/worst 또는 A/B/C)
- `metrics[]` (runway, cost, latency, risk score 등)
- `decisions[]` (선택, 이유, 반려안)

### 3.2 Event Sourcing (권장)

- `TranscriptEvent`
- `WorldUpdateEvent`
- `AgentActionEvent` (interrupt, question, counterfactual)
- `RenderEvent`

---

## 4) MVP 범위 고정 (3주 내 완주용)

> "제품" 만들지 말고 "결정의 순간"을 만든다.

### 반드시 구현

1. **Voice loop**: mic → transcript → agent response(voice or text+voice)
2. **Interrupt 1회**: AI가 조건 충족 시 회의 흐름에 끼어듦
3. **Live Canvas 1종**:
    - (추천) **Decision Tree + Scenario Compare**
4. **Counterfactual 강제**: 항상 대안 2개 생성
5. **Decision Memory 최소**: 세션 로그 JSON 저장

### 절대 제외 (MVP에 넣으면 실패 확률 급상승)

- 하드웨어/물리/CFD/열/응력 "진짜" 시뮬레이터
- 3D 모델링 파이프라인 전체
- 완성형 멀티도메인 통합

## 5) “텍스트로 불가능”을 증명하는 출력 형태 (MVP 추천)

**Live Decision Canvas (single screen)**

- 좌측: 실시간 transcript (speaker diarization)
- 중앙: **Scenario Tree** (Option A/B/C → metric 변화)
- 우측: **Assumption Panel** (가정/근거/신뢰도 + AI가 공격하는 지점)
- 하단: **Diff View** (A vs B 비용/리스크/성능 비교)

이 한 화면이면 “텍스트 챗봇이 아니다”가 즉시 증명된다. (UX 40%에서 바로 먹힌다) Gemini Live Agent Challenge: Re...

---

## 6) GCP 배포 블루프린트 (최소)

- Frontend: Web app (Next.js 등)
- Backend: **Cloud Run** (API + session manager)
- (선택) Firestore: session logs 저장
- Gemini/ADK: backend에서 호출/스트리밍
- 증빙: Cloud Run 서비스 화면/로그 또는 Vertex API 호출 코드로 제출 Gemini Live Agent Challenge: Re...

---

## 7) 리포지토리 구조 (심사자 친화)

- `/apps/web` (UI)
- `/services/api` (Cloud Run)
- `/packages/core`
  - `worldstate/` (schema, validation)
  - `compiler/` (interpretation)
  - `agent/` (interrupt rules, counterfactual)
  - `render/` (diagram spec generators)
- `/infra/` (선택: IaC, bonus) Gemini Live Agent Challenge: Re...
- `README.md`
  - local run

- cloud deploy
- demo script
- test credentials(필요시) Gemini Live Agent Challenge: Re...

---

## 8) 데모 시나리오 (4분 내 완전 승부용)

규정상 "실제 작동"이 핵심.
Gemini Live Agent Challenge: Re...

**Scene 1 (0:00–0:40)**

- "오늘 회의에서 옵션 A/B 중 결정한다"
- 사람 1~2명 음성 입력

**Scene 2 (0:40–1:40)**

- co-founder가 **interrupt**
- "핵심 가정 2개가 위험, 대안 2개 생성하겠다"

**Scene 3 (1:40–3:10)**

- Canvas에 Scenario Tree 생성
- A/B/C 비교 그래프 갱신

**Scene 4 (3:10–4:00)**

- Decision log 저장
- "이 세션은 재실행 가능하다" 보여주고 종료

---

## 9) 보너스 점수(가능하면 얹기)

- 빌드 과정 콘텐츠 1개(공개) + 해시태그 조건 Gemini Live Agent Challenge: Re...
- IaC/스크립트로 배포 자동화 Gemini Live Agent Challenge: Re...
- GDG 프로필 링크 Gemini Live Agent Challenge: Re...

---

## 10) 지금 당장 네가 확정해야 하는 3가지(구조 고정)

1. **Interrupt Policy**: AI가 언제 끼어드는가?
    - 예: "리스크/비용/일정" 키워드 + 불확실성 증가 시

2. **WorldState 1차 도메인**:
    - (추천) "전략 옵션 비교 + 비용/리스크/타임라인"
3. **Canvas 1개 선택**:
    - Decision Tree / System Architecture Mermaid / Cost-Risk Frontier 중 하나

# ✅ PROJECT_SPEC.md

---

# co-founder — Live AI Decision Participant

## 1. Product Essence

co-founder는 C-level 회의에 실제 참가자로 동작하는 AI 시스템이다.

이 시스템은 회의 대화를 입력으로 받아:

- 자연어를 구조화된 world state로 변환하고
- 실시간으로 전략 모델을 생성하며
- 대안 및 반론을 자동 제시하고
- 시각적 구조로 의사결정을 지원한다.

co-founder는 텍스트 응답이 아니라:

👉 **실시간 계산 가능한 모델 + 시각화된 결과물**을 기본 출력으로 한다.

---

## 2. Hackathon Compliance Requirements

프로젝트는 다음 요구사항을 반드시 충족해야 한다:

### Mandatory

- NEW project (기존 코드 재사용 금지)
- Gemini model 사용
- Google GenAI SDK 또는 ADK 사용
- Google Cloud에서 backend 실행
- Real-time multimodal agent (voice interaction 포함)

### Submission artifacts

- public repository
- architecture diagram
- deployment proof
- demo video (실제 작동)

---

## 3. Core Product Concept

### Interaction Model

Human meeting conversation → co-founder listens → co-founder intervenes → visual decision world generated.

co-founder는 다음 역할을 수행한다:

- AI participant
- counterfactual generator
- decision challenger
- visualization engine

---

## 4. Architecture Overview

co-founder는 5-layer architecture를 사용한다.

---

### Layer 1 — Input Layer

Purpose:

회의 입력 수집.

Inputs:

- microphone audio
- streaming transcript

Outputs:

```
TranscriptEvent
{
  timestamp,
```

```
  speaker,
  text
}
```

Responsibilities:

- speech-to-text
- speaker segmentation

---

## Layer 2 — Interpretation Layer (World Compiler)

Purpose:

자연어 → world state.

WorldState structure:

```
WorldState
{
  entities[],
  assumptions[],
  constraints[],
  options[],
  scenarios[],
  metrics[],
  decisions[]
}
```

Key idea:

LLM은 텍스트 생성기가 아니라 world state generator이다.

---

## Layer 3 — AI Participant Engine

co-founder는 단일 AI loop 기반으로 시작한다.

Agent behaviours:

- interrupt meeting
- generate counterfactual scenarios
- propose alternative strategies

- ask critical questions

Output:

```
AgentAction
{
  interrupt: boolean,
  counterfactuals[],
  questions[]
}
```

---

### Layer 4 — Visualization Engine

WorldState → visual model.

Supported outputs (MVP):

- decision tree
- strategy graph
- architecture diagram

Rendering method:

- graph spec (mermaid or graph JSON)

---

### Layer 5 — Decision Memory

Purpose:

회의 의사결정 기록.

Stored:

- assumptions
- rejected options
- chosen decisions
- reasoning history

---

## 5. MVP Scope (Hackathon)

Focus:

Live AI participant experience.

Included:

- voice input
- real-time AI intervention
- live decision visualization
- counterfactual generation

Excluded:

- hardware physics simulation
- 3D modeling
- advanced domain simulation

---

## 6. System Flow

```
Voice Input
   ↓
Speech-to-text
   ↓
World Compiler (Gemini)
   ↓
AI Participant Behaviour
   ↓
WorldState Update
   ↓
Visualization Engine
   ↓
Live Canvas
```

---

## 7. Technical Stack

Frontend:

- web-based UI

Backend:

- Google Cloud Run

AI:

- Gemini (GenAI SDK or ADK)

Storage:

- world state JSON

---

## 8. Demo Scenario

1. Human proposes strategy
2. co-founder listens
3. co-founder interrupts
4. alternative scenario generated
5. visual model appears
6. decision log saved

---

## 9. Long-term Extensibility

Future:

- multi-agent orchestration
- domain simulations
- hardware modeling
- integration with external systems

---

## 10. Design Principles

- conversation-first
- world-state driven
- visualization over text
- mandatory counterfactuals
- real-time interaction

---

# END SPEC

## 🔥 매우 중요한 말 하나 (진짜 핵심)

이 문서 구조는:

👉 Codex / Antigravity / Cursor / 어떤 AI dev agent든

읽으면:

- 프로젝트 구조 생성 가능
- architecture 이해 가능
- 구현 시작 가능